

Code Complexity Analysis - Range Search

Table of Contents

1	Pseudocode	2
1.1	Function 1: rangeSearch	2
1.2	Function 2: rangeQueryRecursive	2
1.3	Function 3: passesFilters	3
2	Time Complexity Analysis	4
2.1	Function: rangeSearch	4
2.2	Function: rangeQueryRecursive	4
2.3	Function: passesFilters	5
3	Mathematical Explanation	6
4	Experimental Example	8

1 Pseudocode

1.1 Function 1: rangeSearch

Location: KD-Tree Implementation

```
1 FUNCTION rangeSearch(latMin, latMax, lonMin, lonMax, isCity, indexCountry,
2   isMainStation):
3   IF root IS NULL:
4     THROW "Root is null"
5
6   results = empty list of Station
7   rangeQueryRecursive(results, root, latMin, latMax, lonMin, lonMax, 0,
8   isCity, indexCountry, isMainStation)
9   RETURN results
10 END FUNCTION
```

1.2 Function 2: rangeQueryRecursive

Location: KD-Tree Implementation

```
1 FUNCTION rangeQueryRecursive(results, node, latMin, latMax, lonMin, lonMax,
2   depth, isCity, indexCountry, isMainStation):
3   IF node IS NULL:
4     RETURN
5
6   lat = node.element.coordinate.latitude
7   lon = node.element.coordinate.longitude
8
9   IF lat BETWEEN latMin AND latMax AND lon BETWEEN lonMin AND lonMax:
10     passesFilters(results, node.element.stations, isCity, indexCountry,
11     isMainStation)
12
13   coord = lon IF depth MOD 2 == 0 ELSE lat
14   minRange = lonMin IF depth MOD 2 == 0 ELSE latMin
15   maxRange = lonMax IF depth MOD 2 == 0 ELSE latMax
16
17   IF coord < minRange:
18     rangeQueryRecursive(results, node.right, latMin, latMax, lonMin, lonMax,
19     depth + 1, isCity, indexCountry, isMainStation)
20   ELSE IF coord > maxRange:
21     rangeQueryRecursive(results, node.left, latMin, latMax, lonMin, lonMax,
22     depth + 1, isCity, indexCountry, isMainStation)
23   ELSE:
24     rangeQueryRecursive(results, node.left, latMin, latMax, lonMin, lonMax,
25     depth + 1, isCity, indexCountry, isMainStation)
26     rangeQueryRecursive(results, node.right, latMin, latMax, lonMin, lonMax,
27     depth + 1, isCity, indexCountry, isMainStation)
28 END FUNCTION
```

1.3 Function 3: passesFilters

Location: KD-Tree Implementation

```
1 FUNCTION passesFilters(filteredStations, candidateStations, isCity,  
  indexCountry, isMainStation):  
2   FOR each station IN candidateStations:  
3     passes = TRUE  
4  
5     IF isCity:  
6       passes = station.isCity()  
7  
8     IF isMainStation:  
9       passes = passes AND station.isMainStation()  
10  
11    SWITCH indexCountry:  
12      CASE 0:  
13        passes = passes AND (station.country.abbreviation == "PT")  
14      CASE 1:  
15        passes = passes AND (station.country.abbreviation == "ES")  
16      DEFAULT:  
17        (do nothing)  
18  
19    IF passes:  
20      filteredStations.add(station)  
21  END FOR  
22 END FUNCTION
```

2 Time Complexity Analysis

2.1 Function: rangeSearch

Operation (pseudocode)	Description	Best Case	Average Case	Worst Case	Notes
<code>results = empty list of Station</code>	Initialize results list	$O(1)$	$O(1)$	$O(1)$	Simple memory allocation
<code>rangeQueryRecursive(root, latMin, latMax, lonMin, lonMax, 0, isCity, indexCountry, isMainStation)</code>	Start recursive range search	$O(\log n)$	$O(\sqrt{n} + k \cdot m)$	$O(n \cdot m)$	n = number of nodes, k = nodes inside range, m = stations per node
<code>RETURN results</code>	Return the final list	$O(1)$	$O(1)$	$O(1)$	Already constructed list
TOTAL		$O(\log n)$	$O(\sqrt{n} + k \cdot m)$	$O(n \cdot m)$	Includes filtering cost

2.2 Function: rangeQueryRecursive

Operation (pseudocode)	Description	Best Case	Average Case	Worst Case	Notes
<code>IF node IS NULL: RETURN</code>	Base case of recursion	$O(1)$	$O(1)$	$O(1)$	Stops recursion for null nodes
<code>lat = node.element.coord lon = node.element.coord</code>	Extract coordinates	$O(1)$	$O(1)$	$O(1)$	Simple assignment
<code>IF lat BETWEEN latMin AND latMax AND lon BETWEEN lonMin AND lonMax: passesFilters(...)</code>	Filter stations in range	$O(m)$	$O(m)$	$O(m)$	Only called nodes inside range; m = stations per node
<code>coord = lon IF depth MOD 2 == 0 ELSE lat minRange = lonMin IF depth MOD 2 == 0 ELSE latMin maxRange = lonMax IF depth MOD 2 == 0 ELSE latMax</code>	Select splitting dimension	$O(1)$	$O(1)$	$O(1)$	Alternates between longitude and latitude

Recursive calls: rangeQueryRecursive(node.left, ...) rangeQueryRecursive(node.right, ...)	Traverse tree	$O(\log n)$	$O(\sqrt{n})$	$O(n)$	$\sqrt{n}$ expected for balanced k-d tree; worst-case degenerate
TOTAL		$O(\log n + k \cdot m)$	$O(\sqrt{n} + k \cdot m)$	$O(n \cdot m)$	$k = \text{nodes in range, } m = \text{stations per node}$

2.3 Function: passesFilters

Operation (pseudocode)	Description	Best Case	Average Case	Worst Case	Notes
FOR each station IN candidateStations:	Loop over all stations in node	$O(m)$	$O(m)$	$O(m)$	$m = \text{stations per node}$
IF isCity: passes = station.isCity() IF isMainStation: passes = passes AND station.isMainStation SWITCH indexCountry: ...	Apply filtering conditions	$O(1)$	$O(1)$	$O(1)$	Constant-time checks per station
IF passes: filteredStations.add(station)	Add station to results	$O(1)$	$O(1)$	$O(1)$	Constant-time insertion per station
TOTAL per node		$O(m)$	$O(m)$	$O(m)$	Linear in stations per node

3 Mathematical Explanation

KD-Tree Range Query Recurrence Explanation

We want to count the number of **regions intersected by a vertical line** in a 2D kd-tree, which is important to understand the range query complexity.

1. The naive guess (incorrect)

At first glance, one might think:

↳ A vertical line intersects either the left or right child of the root, not both.

This would suggest the recurrence:

$$Q(n) = 1 + Q(n/2)$$

But this is incorrect.

- The root splits vertically, but the next level splits horizontally.
- If the vertical line intersects a horizontal split, it may intersect both children of that node.
- Therefore, the situation at the next level is not the same as at the root, so we need to adjust the recursion.

2. Correct approach

- Redefine $Q(n)$ as the number of intersected regions in a subtree whose root is a vertical split.
- Look **two levels down**:
 - After 2 levels, there are 4 regions (each with about $n/4$ points).
 - A vertical line intersects 2 of these 4 regions.
 - Adding the root and first child intersected gives the constant 2.
 - Recursively count the intersected subtrees.

3. Correct recurrence

$$Q(n) = O(1), \text{ if } n = 1$$

$$Q(n) = 2 + 2Q(n/4), \text{ if } n > 1$$

- **2** → regions intersected at the current 2 levels
- **2 $Q(n/4)$** → recursively intersected regions in the two subtrees

4. Expanding the recurrence

$$\begin{aligned}
 Q(n) &= 2 + 2Q(n/4) \\
 &= 2 + 2(2 + 2Q(n/16)) \\
 &= 2 + 4 + 4Q(n/16) \\
 &= 6 + 4Q(n/16) \\
 &\dots
 \end{aligned}$$

After k steps:

$$Q(n) = 2 \cdot (2^k - 1) + 2^k Q(n/4^k)$$

5. Stopping condition

Recursion stops when the subproblem has size 1 (tree size = 1):

$$n/4^k = 1 \Rightarrow 4^k = n \Rightarrow k = \log_4(n)$$

6. Solving the sum

$$\begin{aligned}
 Q(n) &= 2 \cdot (2^k - 1) \\
 &= 2 \cdot (2^{\log_4(n)} - 1) \\
 &= 2 \cdot (n^{1/2} - 1) \\
 &= O(\sqrt{n})
 \end{aligned}$$

- So a vertical line intersects $O(\sqrt{n})$ regions.
- Similarly, a horizontal line intersects $O(\sqrt{n})$ regions.
- Therefore, the total number of nodes visited by a rectangular query is also $O(\sqrt{n})$.

Conclusion and Reference

Conclusion: Even though a kd-tree node may store only a single point, its implicit region is the space defined by the splits from the root. By counting nodes intersected recursively, we obtain the classic 2D range query complexity:

$$T(n) = O(\sqrt{n} + k)$$

where k is the number of reported points.

Reference:

Berg, M. de, Cheong, O., van Kreveld, M., & Overmars, M. (2008). *Computational Geometry: Algorithms and Applications* (3rd edition). Springer-Verlag.
 - Specifically, Section 5.2 ("Kd-trees") and Lemma 5.4 discuss range queries and the number of regions intersected by a line.

4 Experimental Example

Experimental Validation of KD-Tree Range Query

```
1 // Test for average-case behavior of KD-Tree
2 @Test
3 public void test64kAverageCase() {
4     // --- Interval for the query ---
5     double latMin = 40.0;
6     double latMax = 50.0;
7     double lonMin = -10.0;
8     double lonMax = 10.0;
9
10    // --- Filters ---
11    boolean cityFilter = false;
12    boolean mainFilter = false;
13    int countryFilter = 2; // ALL countries
14
15    // --- Reset visited nodes counter ---
16    tree64k.resetVisitedNodes();
17
18    // --- Perform range search ---
19    List<Station> result = tree64k.rangeSearch(
20        latMin, latMax, lonMin, lonMax,
21        cityFilter, countryFilter, mainFilter
22    );
23
24    // --- Gather metrics ---
25    int n = 61163; // total nodes in the tree
26    int k = result.size(); // nodes actually inside the query interval
27    int visited = tree64k.getVisitedNodes(); // total nodes traversed
28
29    // --- Output for verification ---
30    System.out.println("n = " + n);
31    System.out.println("k = " + k);
32    System.out.println("visitedNodes = " + visited);
33    System.out.println("sqrt(n) + k = " + ((int)Math.sqrt(n) + k));
34
35    // --- Assertion: visited nodes should not exceed theoretical bound by too much ---
36    assertTrue(visited <= Math.sqrt(n) + k * 2);
37 }
```

Results:

```
n = 61163
k = 34421
visitedNodes = 34766
sqrt(n) + k (teórico) = 34668
```

The test confirms that the average-case performance of a 2D kd-tree range query matches the theoretical bound $O(\sqrt{n} + k)$. For a tree with 61,163 nodes and 34,421 points returned, the number of nodes visited (34,766) is very close to the theoretical estimate ($\sqrt{n} + k \approx 34,668$). This shows that even for large datasets, the kd-tree efficiently limits the number of nodes traversed, validating the expected asymptotic behavior.